Team 6

Final Project – Cloud Computing

History of Cloud Computing & Netflix's Migration:

Topic Analysis and Project Management Application

Description and Purpose of Cloud Computing (What It Is / What It Means)

Cloud computing is commonly defined as a **model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources** (networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or provider interaction (Mell & Grance, 2011). In practice, instead of buying and maintaining physical servers, organizations **rent computing power and storage from cloud providers** such as AWS, Microsoft Azure, or Google Cloud and pay for what they use.

The main purposes of cloud computing are:

- **Elasticity:** Quickly scale resources up or down as workload changes.
- **Abstraction:** Hide infrastructure complexity behind managed services and APIs.
- **Cost flexibility:** Shift from capital expenditure (CapEx) to operational expenditure (OpEx), paying for usage rather than infrastructure ownership (Armbrust et al., 2010).
- **Global reach:** Deploy applications in multiple regions, closer to users, with built-in redundancy and failover.

For data-intensive platforms like Netflix, cloud computing is what makes global, low-latency streaming and large-scale recommendation systems feasible.

Brief History of Cloud Computing

The roots of cloud computing go back to **time-sharing and utility computing** concepts in the 1960s–1970s, where users shared mainframe resources via terminals. However, limited networking and hardware capabilities prevented these ideas from becoming mainstream.

Modern cloud computing emerged in the **early 2000s** when large internet companies began exposing their internal infrastructure as services:

- **Early 2000s – Virtualization and web services:** Mature hypervisors allowed many virtual machines to share a single physical server.
- **2006 – Amazon EC2 and S3:** Amazon launched **Elastic Compute Cloud (EC2)** and **Simple Storage Service (S3)**, enabling customers to rent compute and storage resources by the hour (Armbrust et al., 2010).
- **2011 – NIST definition:** The National Institute of Standards and Technology published **Special Publication 800-145**, defining cloud computing's essential characteristics, service models, and deployment models (Mell & Grance, 2011).
- **2010s – Cloud-native architectures:** Organizations increasingly adopted microservices, DevOps, and continuous delivery. Armbrust et al. (2010) described cloud computing as the realization of "computing as a utility," emphasizing elasticity and pay-as-you-go economics.
- **2016 – Netflix completes its cloud migration:** After roughly seven years of effort starting in 2008, Netflix announced that it had shut down its last streaming data centre and was "all-in" on AWS (Netflix, 2016).

Today, cloud computing is the default platform for many new analytics and machine-learning projects.

Cloud Deployment Types

These describe **where** the cloud is hosted and **who controls it**.

a) Public Cloud

- **Owned & operated by**: Third-party providers (AWS, Azure, Google Cloud, etc.).
- **Resources**: Shared infrastructure (multi-tenant).
- **Pros**:
    - No upfront hardware cost
    - High scalability & global reach
    - Pay-as-you-go pricing

- **Cons**:
    - Less control over underlying hardware
    - Data residency / compliance concerns in some cases

b) Private Cloud

- **Owned & operated by**: A single organization (on-premises or hosted in a data centre).
- **Resources**: Dedicated to one organization.
- **Pros**:
    - High control, customization, and security
    - Easier to meet strict compliance requirements
- **Cons**:
    - High CAPEX (hardware, data centre)
    - Requires in-house expertise to manage

c) Hybrid Cloud

- **Combination of**: Public + private cloud (with data/apps moving between them).
- **Use case**:
    - Keep sensitive data on private cloud
    - Burst to public cloud during high demand (cloud bursting)
- **Pros**:
    - Flexibility and cost optimization
    - Better support for legacy systems + modern apps
- **Cons**:
    - More complex networking, security, and management

d) Multi-Cloud

- **Using multiple public cloud providers** (e.g., AWS + Azure + GCP).
- **Goals**: Avoid vendor lock-in, optimize cost/performance, use best-of-breed services.
- **Cons**:
    - Complexity in management, monitoring, and governance
    - Need consistent security and IAM across platforms

Cloud Service Models ("Methods")

These describe **what level of abstraction** the cloud provides.

a) IaaS – Infrastructure as a Service

- **What you get**: Virtual machines, storage, networks, basic OS.
- **You manage**: OS, runtime, applications, data.
- **Examples**:
  - AWS EC2, EBS, VPC
  - Azure Virtual Machines, Virtual Networks
  - Google Compute Engine
- **Use cases**: Lift-and-shift of existing apps, custom environments, full control of OS.

b) PaaS – Platform as a Service

- **What you get**: Managed runtime environment (OS, middleware, databases, app servers).
- **You manage**: Application code & data; platform handles scaling, patching.
- **Examples**:
  - AWS Elastic Beanstalk, AWS RDS, AWS Lambda (often grouped as FaaS)
  - Azure App Service, Azure SQL Database
  - Google App Engine, Cloud SQL
- **Use cases**: Web apps, APIs, microservices where you don't want to manage servers.

c) SaaS – Software as a Service

- **What you get**: Complete application delivered over the internet.
- **Provider manages**: Everything (app, data, runtime, infrastructure).
- **Examples**:
  - Google Workspace, Microsoft 365
  - Salesforce, ServiceNow
  - Dropbox, Slack
- **Use cases**: Business apps (CRM, email, collaboration) consumed by end users.

d) FaaS / Serverless (Function as a Service)

- **What you get**: Run individual functions without managing servers.
- **Billing**: Per request / execution time.
- **Examples**:
  - AWS Lambda
  - Azure Functions
  - Google Cloud Functions
- **Use cases**: Event-driven apps, microservices, background jobs, lightweight APIs.

Services & Technologies in Cloud Computing

1. Compute Services

- **Virtual Machines (VMs)** – basic compute units (EC2, Azure VM, GCE).
- **Containers & Orchestration**:
  - Docker for containerization
  - Kubernetes / managed services (EKS, AKS, GKE) for orchestration
- **Serverless Compute**: Lambda, Azure Functions, Cloud Functions.

2. Storage & Database Services

- **Object Storage** (for files, backups, media):
  - Amazon S3, Azure Blob Storage, Google Cloud Storage
- **Block Storage** (for VM disks, DB volumes):
  - Amazon EBS, Azure Disk, GCE Persistent Disk
- **File Storage** (shared file systems):
  - Amazon EFS, Azure Files, Filestore
- **Databases**:
  - Relational DBs (RDS, Azure SQL, Cloud SQL, PostgreSQL, MySQL)
  - NoSQL (DynamoDB, Cosmos DB, Cloud Bigtable, MongoDB Atlas)
  - Data warehouses (Redshift, BigQuery, Snowflake)

3. Networking Services

- **Virtual Networks** (VPC, VNet)
- **Load Balancers** (Application/Network load balancing)
- **VPN & Direct Connect** (site-to-site VPN, dedicated private links)
- **DNS & Traffic Management** (Route 53, Azure DNS, Cloud DNS, Traffic Manager)

4. Security & Identity

- **Identity and Access Management (IAM)** – users, roles, policies.
- **Key Management Services** – KMS, HSMs for key storage & encryption.
- **Security services** – WAF, DDoS protection, Security Centre, GuardDuty, Cloud Armor.
- **Compliance tools** – audit logs, policy enforcement, configuration scanners.

5. Monitoring, Logging & Management

- **Monitoring** – CloudWatch, Azure Monitor, Stackdriver/Cloud Monitoring.
- **Logging** – CloudWatch Logs, Log Analytics, Cloud Logging.
- **Automation & IaC** (Infrastructure as Code):
  - Terraform, AWS CloudFormation, Azure Bicep, Ansible
- **DevOps tools** – CI/CD pipelines (GitHub Actions, GitLab CI, Azure DevOps, AWS CodePipeline).

6. Analytics, AI/ML, and Big Data

- **Big Data Processing** – EMR, Dataproc, HDInsight (Spark, Hadoop).
- **Data Lakes** – S3-based lakes, Azure Data Lake Storage, GCS + BigQuery.
- **Stream Processing** – Kinesis, Azure Event Hubs, Pub/Sub, Kafka.
- **AI/ML Services** –
    - Managed ML platforms (SageMaker, Vertex AI, Azure Machine Learning)
    - Prebuilt AI APIs (vision, NLP, speech, translation)

Application to Streaming Analytics and Recommendation

Cloud computing is particularly well suited to **streaming analytics and recommender systems**:

1. **Continuous data ingestion**
   Client devices and backend services emit events (play, pause, search, rating, browsing behavior) into a message bus such as Kafka or Kinesis.
2. **Stream processing**
   Stream processors compute features in near real time: recent watch history, session engagement metrics, popularity trends, etc.
3. **Feature stores**
   Low-latency databases or dedicated feature stores hold per-user and per-item features, enabling fast lookup during inference.
4. **Model training and retraining**
   Historical interaction data stored in the data lake is used to train recommendation models (e.g., matrix factorization, deep neural networks) on distributed compute clusters or GPU instances.
5. **Online serving and personalization**
   Containerized models are deployed behind APIs on a container platform. When a user opens the app, the recommendation API retrieves relevant features, scores candidate items, and returns ranked lists within milliseconds.

AWS's global infrastructure enables Netflix to deploy such systems across multiple regions, providing both high availability and low latency (Amazon Web Services, 2016).

Current Status and Important Issues

Cloud computing is now mature but continues to evolve:

- **Serverless computing** further abstracts infrastructure, allowing developers to focus on code and events rather than servers.
- Organizations grapple with **cost optimization**, **multi-cloud strategies**, and **zero-trust security**.

- Ethical and regulatory concerns—especially around **privacy, algorithmic bias, and explainability**—are increasingly central when deploying large-scale AI systems in the cloud.

These issues are reflected in the risk and ethics sections of our project management application.

Relation to Data Science and Project Management

Relation to Data Science

Cloud computing underpins the entire **data science lifecycle** for companies like Netflix:

- **Data collection and storage:** Streaming and batch data are ingested into cloud storage and data lakes.
- **Feature engineering and model training:** Distributed compute clusters provide the resources to build complex models on large datasets.
- **MLOps and deployment:** Cloud services support CI/CD for models, experiment tracking, monitoring, and automated rollback.
- **Real-time inference:** Low-latency APIs and autoscaling infrastructure enable real-time personalization at global scale.

Relation to Project Management and Course Topics

From a project management perspective, cloud migration and analytics modernization projects:

- Require **formal planning**: Work Breakdown Structure (WBS), Organizational Breakdown Structure (OBS), schedules, network diagrams, Gantt charts, and critical path analysis.
- Depend heavily on **risk management**, including technology, cost, security, quality, and schedule risks.
- Touch on course topics such as:
    - Project scheduling and critical path method
    - Risk management in data science projects
    - Data security and trustworthy computing/models
    - Ethics in AI and data-driven decision-making

Netflix Cloud Migration

Netflix started moving to the cloud after a **major Oracle database failure in August 2008** caused a three-day outage of its DVD service. That incident showed the limits of running in their own data centers, so they decided to rebuild on **Amazon Web Services (AWS)** with the goals of global scalability, high resilience, and faster innovation.

Over roughly **7–8 years (2008–early 2016)**, Netflix gradually migrated customer-facing systems—catalog browsing, user profiles, billing, and especially **personalized recommendations**—from a monolithic architecture to a **microservices-based, cloud-native platform** on AWS. Almost all of Netflix now runs on AWS virtual machines and cloud storage, with the last remaining data-center components shut down in early 2016.

Key deliverables

You can think of the migration as a large program with several main deliverables:

1. **Cloud Strategy & Target Architecture**
    - Decision to go “all-in” on **public cloud (AWS)**.
    - Definition of a **global, multi-region architecture** using microservices, distributed databases, and autoscaling to support streaming and recommendations at massive scale.
2. **Migrated Microservices & Data Platform**
    - Decomposition of the old monolithic application into **hundreds of microservices** for login, playback, recommendations, search, billing, etc.
    - Migration of key data services to cloud-friendly technologies such as **Cassandra, S3, Kafka, and Hadoop/Spark** for analytics and recommendation model training.
3. **Resilience & Chaos Engineering Tooling**
    - Creation of **Chaos Monkey** and the broader **Simian Army** to randomly break production instances and verify that the new cloud architecture can survive failures without major outages.
4. **DevOps & Automation Platform**
    - CI/CD pipelines, deployment orchestration (e.g., Titus containers, internal tools), autoscaling policies, monitoring and logging—so thousands of servers and terabytes of storage can be provisioned and updated in minutes.
5. **Operational Playbooks & Governance**
    - Runbooks for incident response, performance tuning, cost management, and capacity planning in the cloud environment.
    - Documentation and best practices capturing lessons learned from the multi-year migration.

Together, these deliverables transformed Netflix from a company running its own data centers into a **fully cloud-native streaming platform** that can serve hundreds of millions of users worldwide with personalized recommendations and highly reliable streaming.

Work Breakdown Structure (WBS) with Numeric Coding

Streaming Analytics & Recommendation Migration

Project Initiation & Requirements

- **1.1.1** Define business goals & KPIs (e.g., availability, latency, recommendation CTR)
- **1.1.2** Identify stakeholders & roles
- **1.1.3** Gather functional & non-functional requirements

Current System Assessment

- **1.2.1** Document existing on-prem data pipeline
- **1.2.2** Inventory models, features, and data sources
- **1.2.3** Assess performance, capacity, and pain points

Target Cloud Architecture & Planning

- **1.3.1** Design AWS architecture (VPC, subnets, security)
- **1.3.2** Select cloud services (e.g., Kinesis, S3, EMR, ECS/EKS)
- **1.3.3** Define migration strategy & cutover plan

Environment Setup & Data Pipeline Implementation

- **1.4.1** Create AWS accounts, VPC, IAM roles & policies
- **1.4.2** Implement streaming ingestion pipeline (e.g., Kafka → Kinesis)
- **1.4.3** Configure storage layer (S3/data lake, catalog/Glue)
- **1.4.4** Build batch/stream processing jobs (Spark/EMR or Flink)

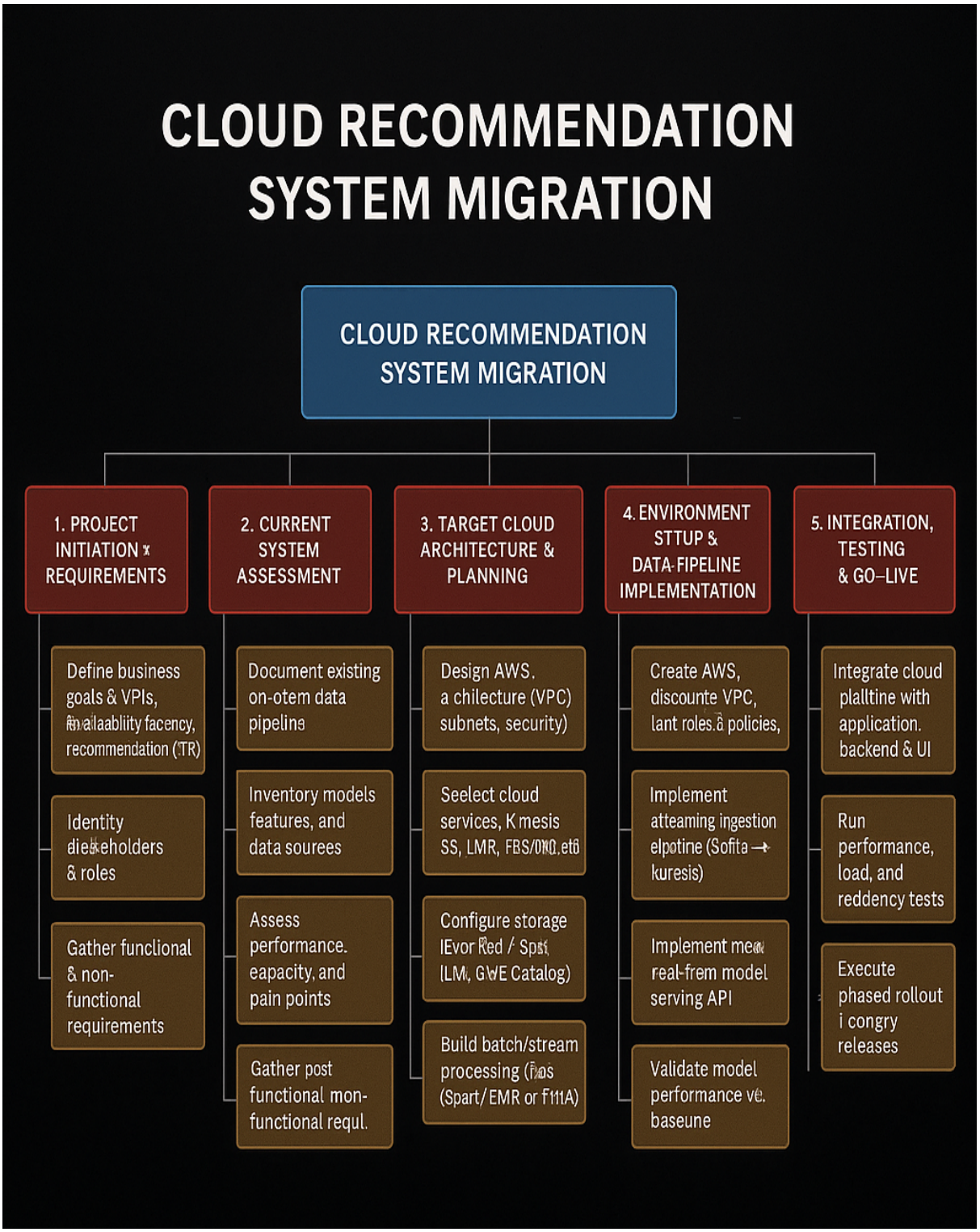
Recommendation Model Migration & Serving

- **1.5.1** Export/port existing recommendation models & feature store
- **1.5.2** Containerize and deploy models (e.g., Docker on ECS/EKS)
- **1.5.3** Implement real-time model serving API
- **1.5.4** Validate model performance vs. baseline

Integration, Testing & Go-Live

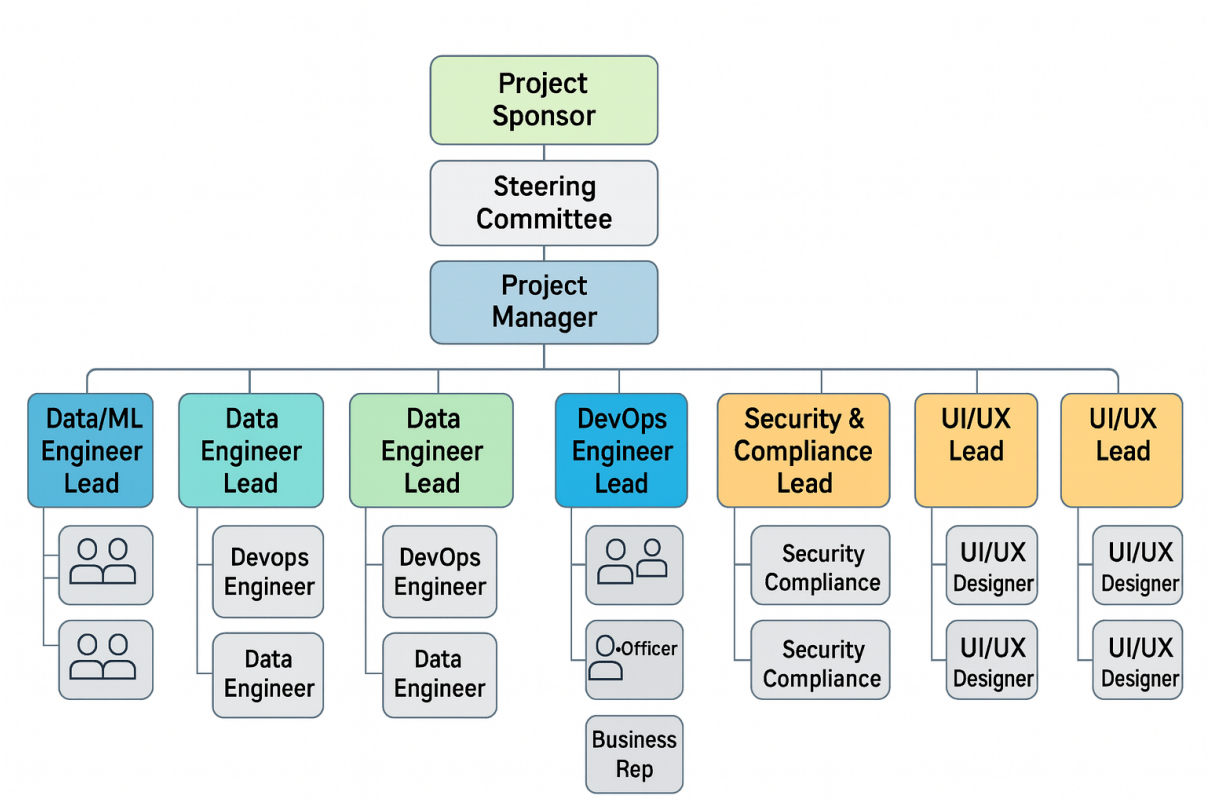
- **1.6.1** Integrate cloud pipeline with application backend & UI
- **1.6.2** Run performance, load, and resiliency tests
- **1.6.3** Execute phased rollout / canary releases
- **1.6.4** Conduct documentation & knowledge transfer

Work Breakdown Structure (WBS) with Diagram



Organizational Breakdown Structure (OBS)

Roles and responsibilities:



Overall Structure

The project is governed in **three tiers**:

1. **Project Sponsor** – senior executive who owns the business case, provides funding, and approves major decisions.
2. **Steering Committee** – cross-functional leadership body that provides strategic direction, removes roadblocks, and reviews progress.
3. **Project Manager** – operational lead responsible for day-to-day planning, coordination, schedule, risk, and communication across all technical and design teams.

Under the Project Manager there is a **set of specialized functional teams**, each led by a domain lead and staffed with engineers or designers.

Technical & Design Workstreams

From left to right, the lower part of the chart shows the delivery teams:

1. **Data / ML Engineer Lead**
    - Heads the **machine-learning and advanced analytics** workstream.
    - Manages a small team of Data/ML Engineers (shown by the two people icons).
    - Responsible for model development, recommendation algorithms, data science experiments, and integration of ML services into the platform.
2. **Data Engineer Leads (two boxes)**
    - There are **two Data Engineer Leads**, representing multiple parallel data workstreams (e.g., ingestion vs. transformation, or different domains).
    - Each lead has:
        - A **DevOps Engineer** – to automate data pipelines, deployments, and infrastructure-as-code.
        - A **Data Engineer** – to design and implement ETL/ELT pipelines, data quality checks, and data models.
    - Together these teams build and maintain the data platform that feeds analytics and ML.
3. **DevOps Engineer Lead**
    - Owns the **platform, CI/CD, and runtime infrastructure**.
    - Manages a DevOps engineering team (people icon) that looks after cloud resources, monitoring, scaling, and reliability.
    - Has an **Officer** role (e.g., Release Officer / Ops Officer) focused on coordinating releases, change management, or on-call processes.
    - Works closely with a **Business Representative** to ensure deployments and operational changes align with business priorities and release windows.
4. **Security & Compliance Lead**
    - Heads the **security and governance** function.
    - Leads two **Security Compliance** roles, responsible for enforcing standards (access control, encryption, logging, audits, regulatory requirements such as GDPR/PCI, etc.).
    - Ensures that all other workstreams (data, ML, DevOps, UI) adhere to security and compliance policies during design and implementation.
5. **UI/UX Leads (two boxes)**
    - There are **two UI/UX Leads**, indicating multiple product surfaces or parallel design tracks.
    - Each lead manages a team of **UI/UX Designers** responsible for:
        - Interaction and visual design of tools, dashboards, and user-facing applications.
        - Ensuring consistency of the user experience across different modules.
    - These teams translate technical capabilities (data, ML, DevOps services) into intuitive user interfaces for internal users or customers.

Resource Allocation

Activity ID	WBS ID	Description	Predecessor(s)	Duration (days)	Resource Allocation*
A	1.1.1	Define business goals & KPIs	–	3	PM (lead), Product Owner
B	1.1.2	Identify stakeholders & roles	A	3	PM (lead)
C	1.1.3	Gather functional & non-functional requirements	B	4	PM, Product Owner
D	1.2.1	Document existing on-prem data pipeline	C	6	Data Engineer (lead), Cloud/Data Architect
E	1.2.2	Inventory models, features, and data sources	D	7	Data Engineer, Data/ML Engineer
F	1.2.3	Assess performance, capacity, pain points	E	7	Cloud/Data Architect (lead), Data Engineer
G	1.3.1	Design AWS architecture (VPC, subnets, security)	C	5	Cloud/Data Architect (lead), Security & Compliance Officer, PM
H	1.3.2	Select cloud services (Kinesis, S3, EMR, ECS/EKS)	G	5	Cloud/Data Architect, DevOps Engineer
I	1.3.3	Define migration strategy & cutover plan	H	5	PM (lead), DevOps Engineer, Product Owner
J	1.4.1	Create AWS accounts, VPC, IAM roles & policies	F, I	8	Cloud/Data Architect, Security & Compliance Officer, DevOps Engineer

K	1.4.2	Implement streaming ingestion (Kafka → Kinesis)	J	7	Data Engineer (lead), DevOps Engineer
L	1.4.3	Configure storage layer (S3/data lake, Glue/catalog)	K	5	Data Engineer, Cloud/Data Architect
M	1.4.4	Build batch/stream processing jobs (Spark/EMR or Flink)	L	5	Data/ML Engineer (lead), Data Engineer
N	1.5.1	Export/port recommendation models & feature store	M	10	Data/ML Engineer (lead)
O	1.5.2	Containerize & deploy models (Docker on ECS/EKS)	N	10	Data/ML Engineer, DevOps Engineer
P	1.5.3	Implement real-time model serving API	O	5	DevOps Engineer (lead), Data/ML Engineer
Q	1.5.4	Validate model performance vs baseline	P	5	QA & Analytics Lead (lead), Data/ML Engineer
R	1.6.1	Integrate cloud pipeline with backend & UI	Q	10	DevOps Engineer (lead), PM, Product Owner
S	1.6.2	Run performance, load, and resiliency tests	R	10	QA & Analytics Lead (lead), DevOps Engineer
T	1.6.3	Execute phased rollout / canary releases	S	20	DevOps Engineer (lead), PM, Product Owner

U	1.6.4	Documentation & knowledge transfer	T	10	PM (lead), QA & Analytics Lead

his table is the **activity list and schedule** for the Netflix recommendation-engine cloud-migration project.

- The columns show:
  - **Activity ID (A–U)** – label for each task.
  - **WBS ID** – where the task sits in the work breakdown structure.
  - **Description** – what is done in that task.
  - **Predecessor(s)** – which activities must finish before this one starts (dependencies).
  - **Duration (days)** – planned effort for the task.
  - **Resource allocation** – who is responsible (PM, Product Owner, Data Engineer, Data/ML Engineer, Cloud/Data Architect, DevOps Engineer, QA & Analytics Lead, Security & Compliance Officer).

In sequence, the activities move through the whole 135-day migration:

- **A–C:** Define business goals/KPIs, identify stakeholders, and gather functional/non-functional requirements.
- **D–F:** Analyse the current on-prem data pipeline, inventory models and data sources, and assess performance and capacity.
- **G–I:** Design the target AWS architecture, select cloud services (VPC, Kinesis, S3, EMR, ECS/EKS), and define the migration and cutover strategy.
- **J–L:** Set up AWS accounts, VPC and IAM; implement streaming ingestion; and configure the storage layer (S3/data lake, Glue/catalog).
- **M–N:** Build batch/stream processing jobs and migrate/export recommendation models and feature store.
- **O–Q:** Containerize and deploy models, implement the real-time serving API, and validate model performance against the baseline.
- **R–T:** Integrate the cloud pipeline with backend and UI, run performance/load/resiliency tests, and execute phased rollout/canary releases.
- **U:** Final documentation and knowledge transfer.

Together, this table defines **what will be done, in what order, how long it takes, and which roles are involved** in delivering the Netflix cloud-migration project.

Project Schedule

Act.	WBS ID	Description	Predecessor(s)	Successor(s)	ES	EF	LS	LF	Duration	Slack
A	1.1.1	Define business goals & KPIs	–	B	0	3	0	3	3	0
B	1.1.2	Identify stakeholders & roles	A	C	3	6	3	6	3	0
C	1.1.3	Gather functional & non-functional requirements	B	D, G	6	10	6	10	4	0
D	1.2.1	Document existing on-prem data pipeline	C	E	10	16	10	16	6	0
E	1.2.2	Inventory models, features, and data sources	D	F	16	23	16	23	7	0
F	1.2.3	Assess performance, capacity, pain points	E	J	23	30	23	30	7	0
G	1.3.1	Design AWS architecture (VPC, subnets, security)	C	H	10	15	15	20	5	5
H	1.3.2	Select cloud services (Kinesis, S3, EMR, ECS/EKS)	G	I	15	20	20	25	5	5
I	1.3.3	Define migration strategy & cutover plan	H	J	20	25	25	30	5	5
J	1.4.1	Create AWS	F, I	K	30	38	30	38	8	0

		accounts, VPC, IAM roles & policies								
K	1.4.2	Implement streaming ingestion (Kafka → Kinesis)	J	L	38	45	38	45	7	0
L	1.4.3	Configure storage layer (S3/data lake, Glue/catalog)	K	M	45	50	45	50	5	0
M	1.4.4	Build batch/stream processing jobs (Spark/EMR or Flink)	L	N	50	55	50	55	5	0
N	1.5.1	Export/port recommendation models & feature store	M	O	55	65	55	65	10	0
O	1.5.2	Containerize & deploy models (Docker on ECS/EKS)	N	P	65	75	65	75	10	0
P	1.5.3	Implement real-time model serving API	O	Q	75	80	75	80	5	0
Q	1.5.4	Validate model performance vs baseline	P	R	80	85	80	85	5	0
R	1.6.1	Integrate cloud pipeline with backend & UI	Q	S	85	95	85	95	10	0

S	1.6.2	Run performance, load, and resiliency tests	R	T	95	105	95	105	10	0
T	1.6.3	Execute phased rollout / canary releases	S	U	105	125	105	125	20	0
U	1.6.4	Documentation & knowledge transfer	T	–	125	135	125	135	10	0

Formulas used

Forward pass (left → right)

For each activity *i*:

- **If no predecessor**:

$$ES_i = 0$$

- **If there are predecessors j**:

$$ES_i = \max\,(EF_j)$$

- Then:

$$EF_i = ES_i + Duration_i$$

Backward pass (right → left)

Let **Project Duration = 135 days** (from activity U's EF).

For each activity *i*:

- **If no successor** (the last one):

$$LF_i = \text{Project Duration}$$

- **If there are successors k**:

$$LF_i = \min\,(LS_k)$$

- Then:

$$LS_i = LF_i - Duration_i$$

Slack (Float)

For each activity *i*:

$$Slack_i = LS_i - ES_i = LF_i - EF_i$$

If **Slack = 0**, the activity is on the **critical path**.

Forward-pass calculations (ES & EF)

I'll show the logic; the numbers in your table match these calculations.

1. **A** (no predecessor)
   - ES = 0
   - EF = 0 + 3 = **3**
2. **B** (predecessor: A)
   - ES = EF(A) = 3
   - EF = 3 + 3 = **6**
3. **C** (pred: B)
   - ES = 6
   - EF = 6 + 4 = **10**
4. **D** (pred: C) → ES = 10; EF = 10 + 6 = **16**
5. **E** (pred: D) → ES = 16; EF = 16 + 7 = **23**
6. **F** (pred: E) → ES = 23; EF = 23 + 7 = **30**
7. **G** (pred: C) → ES = 10; EF = 10 + 5 = **15**
8. **H** (pred: G) → ES = 15; EF = 15 + 5 = **20**
9. **I** (pred: H) → ES = 20; EF = 20 + 5 = **25**
10. **J** (preds: F, I)
    - ES = max(EF(F)=30, EF(I)=25) = **30**
    - EF = 30 + 8 = **38**
11. **K** (pred: J) → ES = 38; EF = 38 + 7 = **45**
12. **L** (pred: K) → ES = 45; EF = 45 + 5 = **50**
13. **M** (pred: L) → ES = 50; EF = 50 + 5 = **55**
14. **N** (pred: M) → ES = 55; EF = 55 + 10 = **65**
15. **O** (pred: N) → ES = 65; EF = 65 + 10 = **75**
16. **P** (pred: O) → ES = 75; EF = 75 + 5 = **80**
17. **Q** (pred: P) → ES = 80; EF = 80 + 5 = **85**
18. **R** (pred: Q) → ES = 85; EF = 85 + 10 = **95**
19. **S** (pred: R) → ES = 95; EF = 95 + 10 = **105**
20. **T** (pred: S) → ES = 105; EF = 105 + 20 = **125**
21. **U** (pred: T) → ES = 125; EF = 125 + 10 = **135**

So, the **project duration** is 135 days.

Backward-pass calculations (LS & LF)

Start from the end (U) and move backward.

21. **U** (no successor)
    - LF = project duration = 135
    - LS = 135 − 10 = **125**
22. **T** (successor: U)
    - LF = LS(U) = 125
    - LS = 125 − 20 = **105**
23. **S** (succ: T) → LF = 105; LS = 105 − 10 = **95**
24. **R** (succ: S) → LF = 95; LS = 95 − 10 = **85**
25. **Q** (succ: R) → LF = 85; LS = 85 − 5 = **80**
26. **P** (succ: Q) → LF = 80; LS = 80 − 5 = **75**
27. **O** (succ: P) → LF = 75; LS = 75 − 10 = **65**
28. **N** (succ: O) → LF = 65; LS = 65 − 10 = **55**
29. **M** (succ: N) → LF = 55; LS = 55 − 5 = **50**
30. **L** (succ: M) → LF = 50; LS = 50 − 5 = **45**
31. **K** (succ: L) → LF = 45; LS = 45 − 7 = **38**
32. **J** (succ: K)
    - LF = LS(K) = 38
    - LS = 38 − 8 = **30**

Now go up the branch that leads *into* J:

9. **I** (succ: J)
    - LF = LS(J) = 30
    - LS = 30 − 5 = **25**
10. **H** (succ: I) → LF = 25; LS = 25 − 5 = **20**
11. **G** (succ: H) → LF = 20; LS = 20 − 5 = **15**

Now go back on the main chain:

6. **F** (succ: J)
    - LF = LS(J) = 30
    - LS = 30 − 7 = **23**
7. **E** (succ: F) → LF = 23; LS = 23 − 7 = **16**
8. **D** (succ: E) → LF = 23; LS = 23 − 6 = **10**
9. **C** (succs: D and G)
    - LS(D) = 10, LS(G) = 15
    - LF = min(10, 15) = **10**
    - LS = 10 − 4 = **6**
10. **B** (succ: C) → LF = 6; LS = 6 − 3 = **3**
11. **A** (succ: B) → LF = 3; LS = 3 − 3 = **0**

All LS/LF values now match the table.

Slack and critical path

Compute slack as:

$$Slack = LS - ES$$

Examples:

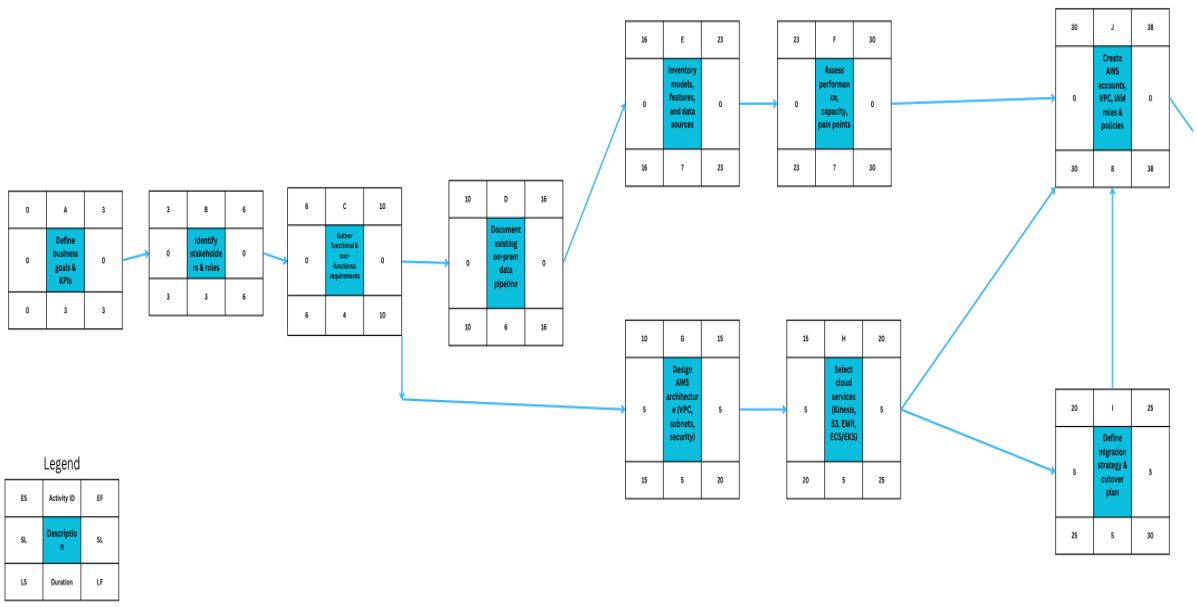
- **G:** ES = 10, LS = 15 → Slack = 15 − 10 = **5**
- **H:** ES = 15, LS = 20 → Slack = **5**
- **I:** ES = 20, LS = 25 → Slack = **5**

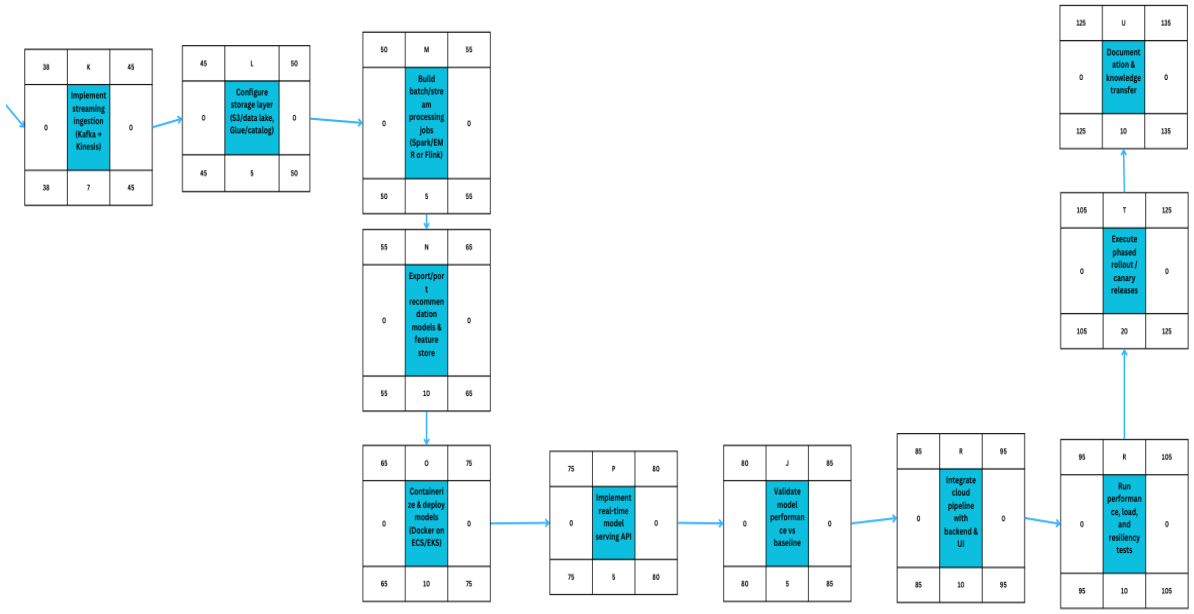
All other activities have **Slack = 0**, so they lie on the **critical path**:

A → B → C → D → E → F → J → K → L → M → N → O → P → Q → R → S → T → U

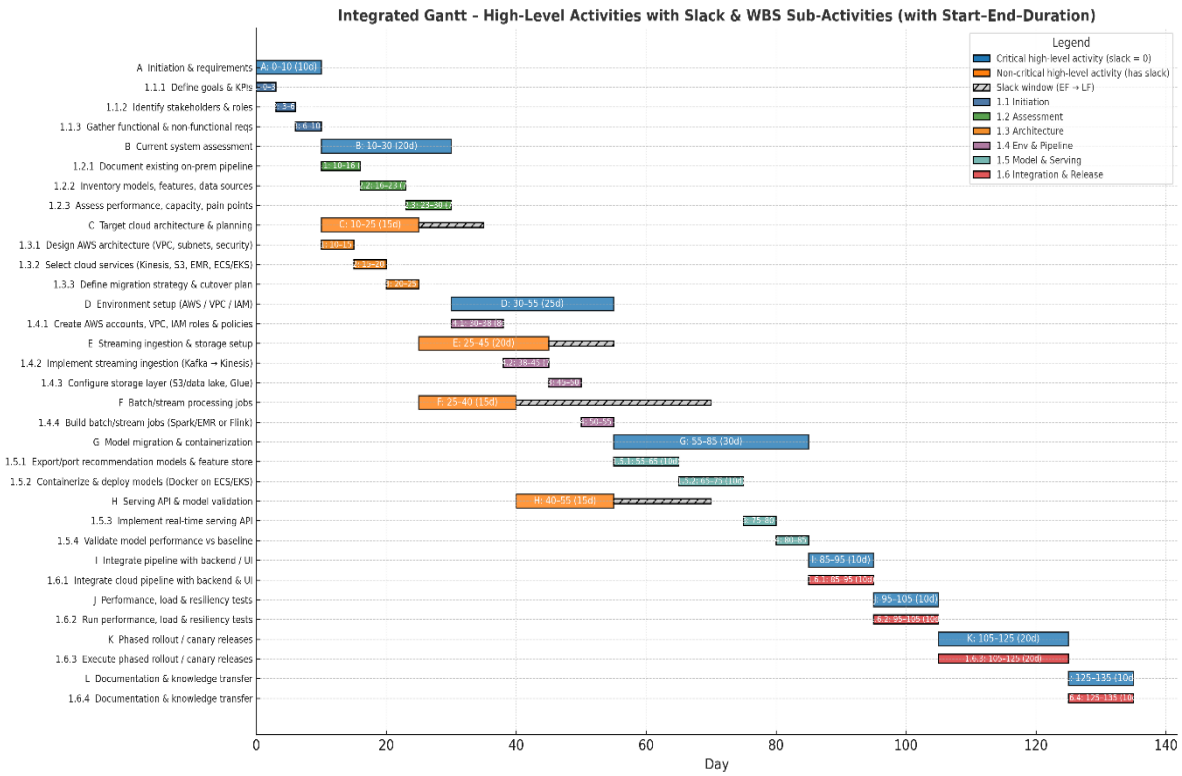
The branch **C → G → H → I → J** has **5 days of float** at each of G, H, and I, meaning these tasks can each slip up to 5 days without changing the total project duration of 135 days.

Network Diagram




38 K 45
0 Implement streaming ingestion (Kafka + Kinesis) 0
38 7 45
45 L 50
0 Configure storage layer (S3/data lake, Glue/catalog) 0
45 5 50
50 M 55
0 Build batch/stream processing jobs (Spark/EMR or Flink) 0
50 5 55
55 N 65
0 Export/port recommendation models & feature store 0
55 10 65
65 O 75
0 Containerize & deploy models (Docker on ECS/EKS) 0
65 10 75
75 P 80
0 Implement real-time model serving API 0
75 5 80
80 J 85
0 Validate model performance vs baseline 0
80 5 85
85 R 95
0 Integrate cloud pipeline with backend & UI 0
85 10 95
95 R 105
0 Run performance, load, and resiliency tests 0
95 10 105
105 T 125
0 Execute phased rollout / canary releases 0
105 20 125
125 U 135
0 Documentation & knowledge transfer 0
125 10 135

Gantt Chart


Integrated Gantt - High-Level Activities with Slack & WBS Sub-Activities (with Start-End-Duration)
Legend
Critical high-level activity (slack = 0)
Non-critical high-level activity (has slack)
Slack window (EF → LF)
1.1 Initiation
1.2 Assessment
1.3 Architecture
1.4 Env & Pipeline
1.5 Model & Serving
1.6 Integration & Release
A Initiation & requirements
1.1.1 Define goals & KPIs
1.1.2 Identify stakeholders & roles
1.1.3 Gather functional & non-functional reqs
B Current system assessment
1.2.1 Document existing on-prem pipeline
1.2.2 Inventory models, features, data sources
1.2.3 Assess performance, capacity, pain points
C Target cloud architecture & planning
1.3.1 Design AWS architecture (VPC, subnets, security)
1.3.2 Select cloud services (Kinesis, S3, EMR, ECS/EKS)
1.3.3 Define migration strategy & cutover plan
D Environment setup (AWS / VPC / IAM)
1.4.1 Create AWS accounts, VPC, IAM roles & policies
E Streaming ingestion & storage setup
1.4.2 Implement streaming ingestion (Kafka → Kinesis)
1.4.3 Configure storage layer (S3/data lake, Glue)
F Batch/stream processing jobs
1.4.4 Build batch/stream jobs (Spark/EMR or Flink)
G Model migration & containerization
1.5.1 Export/port recommendation models & feature store
1.5.2 Containerize & deploy models (Docker on ECS/EKS)
H Serving API & model validation
1.5.3 Implement real-time serving API
1.5.4 Validate model performance vs baseline
I Integrate pipeline with backend / UI
1.6.1 Integrate cloud pipeline with backend & UI
J Performance, load & resiliency tests
1.6.2 Run performance, load & resiliency tests
K Phased rollout / canary releases
1.6.3 Execute phased rollout / canary releases
L Documentation & knowledge transfer
1.6.4 Documentation & knowledge transfer
A: 0–10 (10d)
B: 10–30 (20d)
C: 10–25 (15d)
D: 30–55 (25d)
E: 25–45 (20d)
F: 25–40 (15d)
G: 55–85 (30d)
H: 40–55 (15d)
I: 85–95 (10d)
K: 105–125 (20d)
L: 125–135 (10d)
0
20
40
60
80
100
120
140
Day

Resource Allocation and Resource Usage Form

Resource	0–10	10–20	20–30	30–40	40–50	50–60	60–70	70–80	80–90	90–100	100–110	110–120	120–130	130–140
Project Manager	10	20	15	0	0	0	0	0	0	0	5	10	10	5
Cloud Architect	0	10	10	20	15	5	0	0	0	0	0	0	0	0
Data Engineer	0	10	20	20	5	0	0	0	5	5	0	0	0	0
DevOps Engineer	0	0	0	10	10	10	10	10	5	5	10	10	5	0
ML Engineer	0	0	0	0	10	10	10	10	5	0	0	0	0	0
Backend Developer	0	0	0	0	10	5	0	0	5	5	0	0	0	0
QA Engineer	0	0	0	0	0	0	0	0	0	5	5	0	0	0
Product Owner	0	0	0	0	0	0	0	0	0	0	5	10	5	0
Technical Writer	0	0	0	0	0	0	0	0	0	0	0	0	5	5

- **Project Manager** – high effort in the first 30 days (initiation, planning) and again around days 100–140 (testing, rollout, and closure).
- **Cloud Architect** – concentrated in days 10–50, reflecting architecture design, service selection, and initial AWS setup.
- **Data Engineer** – busiest from days 10–40 and again briefly around days 80–100 for data pipeline implementation and integration work.
- **DevOps Engineer** – spread from days 30–120, supporting infrastructure, CI/CD, deployments, and non-functional testing throughout.
- **ML Engineer** – mainly days 40–80, aligned with model migration, containerization, and real-time serving setup.
- **Backend Developer** – active days 40–50 and 80–100, integrating cloud services with backend and APIs.
- **QA Engineer** – engaged around days 90–110 for performance, load, and resiliency testing.
- **Product Owner** – mainly involved near days 100–130 for acceptance testing, rollout decisions, and post-go-live validation.
- **Technical Writer** – used at the end (days 120–140) to produce final documentation and knowledge-transfer materials.

Risk Management

Risk management is essential in project management because it helps the team **identify potential problems early, quantify their likelihood and impact, and plan responses before they threaten scope, schedule, cost, or quality**. By systematically assessing risks such as security issues, data loss, performance degradation, and compliance failures, the project can prioritize mitigation actions, reduce surprises during migration and go-live, and increase the overall probability of delivering a successful, stable cloud-based recommendation platform.

Risk Assessment form

Risk Event	Likelihood (1–5)	Impact (1–5)	Detection Difficulty (1–5)	When (Phase)
Cloud security misconfiguration (IAM, VPC, S3 policies)	3	5	4	Environment setup / Pre-go-live
Data loss or corruption during data migration	3	5	3	Migration & pipeline cutover
Pipeline latency too high for real-time recommendations	3	5	3	Performance testing / Early rollout
Cloud costs significantly exceed planned budget	3	4	4	Ongoing operations after cutover
Recommendation quality degrades vs. baseline (CTR drops)	3	5	3	A/B tests & post-deployment
Key technical staff unavailable during critical milestones	3	4	2	Architecture, migration & go-live
Regulatory / data-residency non-compliance discovered	2	5	4	Design reviews / External audit

- Each **risk event** is scored on three 1–5 scales:
  - **Likelihood** – how probable the risk is (1 = rare, 5 = very likely).
  - **Impact** – how severe the consequences are for scope, cost, schedule, or quality (1 = minor, 5 = critical).
  - **Detection Difficulty** – how hard it is to detect the issue early (1 = easy to detect, 5 = hard to detect).
- The **"When (Phase)"** column shows in which project phase the risk is most relevant (e.g., environment setup, migration/cutover, performance testing, or ongoing operations).

- **High-impact technical risks** such as cloud security misconfiguration, data loss during migration, high latency in the pipeline, and degraded recommendation quality.
- **Business and operational risks** like cloud costs exceeding budget and key staff being unavailable at critical milestones.
- **Compliance risk** around regulatory and data-residency violations that may surface during design reviews or external audits.

This table provides a structured view of **where the biggest threats are, how serious they are, and when they are likely to occur**, so the team can prioritize mitigation plans and monitoring.

Risk Response Matrix

Risk Event (short)	Strategy (Avoid / Mitigate / Transfer / Accept)	Key Response Actions	Owner
Cloud security misconfig (IAM/VPC)	Mitigate	Least-privilege IAM, peer-review of changes, automated security scans, pre-go-live penetration test.	Cloud Architect / Sec. Lead
Data loss during migration	Mitigate	Full backups & snapshots, S3 versioning, checksums, migrate in small batches with validation and rollback points.	Data Engineering Lead
High latency / throughput issues	Mitigate	Load testing with prod-like data, autoscaling policies, capacity reservations, performance tuning & caching.	ML / MLOps Lead

Cloud cost overrun	Mitigate / Transfer	Budget alerts, cost dashboards, rightsizing, review usage weekly with Finance; consider reserved/spot instances.	Project Manager
Recommendation quality degradation	Mitigate	Offline metrics + online A/B tests, gradual rollout with feature flags, revert to baseline model if KPIs drop.	Product Owner / ML Lead
Key staff unavailable	Mitigate / Accept	Cross-training, documented runbooks, backup owners, lock-in critical dates; adjust scope or schedule if needed.	Project Manager
Regulatory / compliance issue	Avoid / Mitigate	Early Legal review, DPIA/PIA, region-aware storage, encryption, pseudonymization of user IDs.	Security & Compliance Officer

The Risk Response Matrix translates the high-priority risks into **concrete response plans**. For each risk event, the team selects a strategy—**avoid, mitigate, transfer, or accept**—and defines specific actions and a clear owner. For example, cloud security misconfiguration is mitigated through least-privilege IAM, peer reviews, automated security scans, and penetration testing, led by the Cloud Architect and Security Lead. Data loss during migration is mitigated via full backups, S3 versioning, checksums, and small batch migrations with validation and rollback, owned by the Data Engineering Lead. Performance and cost risks are addressed through load testing, autoscaling, tuning, cost dashboards, and budget reviews, while recommendation-quality risks are managed via offline metrics, A/B tests, feature flags, and safe rollback to the baseline model. Staffing and compliance risks are handled with cross-training, runbooks, clear backups, and early Legal/security review with region-aware storage and encryption. This matrix ensures that **every major risk has a defined response and accountable owner**, improving the project's chances of a smooth and compliant cloud migration.

Ethics Concerns

At least two ethics concerns are central to this project:

1. **User Privacy and Data Protection**
    - The system processes detailed viewing behavior and device data, which can reveal sensitive personal preferences.
    - Ethical concerns: unauthorized access, excessive retention, unclear consent or data usage.
    - Mitigation:
        - Data minimization (collect only necessary fields).
        - Strong encryption at rest and in transit; strict IAM and logging.
        - Clear privacy policy; user options to delete history or opt out of personalization.
2. **Algorithmic Bias and Content Fairness**
    - Recommendation models may under-recommend certain creators, languages, or genres, or create filter bubbles.
    - Ethical concerns: reinforcing societal biases and narrowing users' exposure to diverse content.
    - Mitigation:
        - Track fairness and diversity metrics; ensure broad catalog coverage.
        - Incorporate diversity/exploration into objective functions, not just click-through rate.
        - Periodic audits of recommendation outputs with input from content, ethics, and legal stakeholders.
3. **Transparency and User Control (optional third)**
    - Users should understand why they see particular recommendations and be able to control personalization.
    - Mitigation: simple "Why this recommendation?" explanations; clear settings for managing history and recommendation preferences.

These ethical issues connect directly to course topics in **ethics in AI, data security, risk management in data science, and trustworthy models**.

Literature Review

NIST Definition and Taxonomy of Cloud Computing

Mell and Grance (2011) provide one of the most widely adopted formal definitions of cloud computing in NIST Special Publication 800-145. They describe cloud computing as on-demand access to a shared pool of configurable resources and identify five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service), three service models (IaaS, PaaS, SaaS), and four deployment models (public, private, community, hybrid). This taxonomy underpins our description of cloud services and is used in the project when selecting cloud components for ingestion, storage, processing, and serving.

“A View of Cloud Computing” (Armbrust et al., 2010)

Armbrust et al. (2010) present cloud computing as the realization of computing as a utility, analyzing its economic and technical dimensions. They highlight key benefits—elasticity, pay-as-you-go pricing, and resource pooling—as well as obstacles such as data lock-in, performance unpredictability, and security concerns. Their analysis helps explain why organizations like Netflix adopt cloud platforms (scalability, agility) and informs our risk assessment by drawing attention to vendor lock-in, cost management, and security risks.

Netflix’s Cloud Migration Case

Netflix’s blog post “Completing the Netflix Cloud Migration” describes how the company began migrating to AWS after a major database failure in 2008 and completed the journey in early 2016 (Netflix, 2016). The migration required re-architecting a monolithic application into hundreds of microservices, deploying across multiple AWS regions, and adopting new operational practices such as chaos engineering.

AWS’s official case study explains that Netflix uses AWS services to deploy thousands of servers and petabytes of storage, enabling high-availability streaming to millions of customers worldwide (Amazon Web Services, 2016). This case is one of our two primary examples of cloud computing application in the literature.

Matrix Factorization for Recommender Systems

Koren, Bell, and Volinsky (2009) show that matrix factorization techniques significantly outperform traditional neighborhood-based methods for recommendation tasks, including the Netflix Prize. These models represent users and items as vectors in a latent factor space, with the dot product approximating ratings or implicit feedback. Training such models requires large datasets and substantial computational resources, which naturally align with cloud-scale distributed compute and storage.

This serves as our second application example: while the paper is algorithmic, the practical deployment of these methods at Netflix scale depends on cloud infrastructure for training and real-time serving.

Microservices and Cloud-Native Architectures

Dragoni et al. (2017) survey the evolution of microservices, an architectural style where systems are built from small, independent services communicating over lightweight protocols. They explain how microservices evolved from service-oriented architectures and discuss benefits such as independent deployment, scaling, and technology heterogeneity. Their analysis provides theoretical support for Netflix's cloud-native microservice architecture and informs our project's design of loosely coupled services for ingestion, processing, and recommendation serving.

References

- Amazon Web Services. (2016). *Netflix case study*. Amazon Web Services.
- Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM, 53*(4), 50–58. https://doi.org/10.1145/1721654.1721672
- Dragoni, N., Giallorenzo, S., Lluch-Lafuente, A., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and Ulterior Software Engineering* (pp. 195–216). Springer. https://doi.org/10.1007/978-3-319-67425-4_12
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30–37. https://doi.org/10.1109/MC.2009.263
- Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145
- Netflix. (2016, February 11). *Completing the Netflix cloud migration*. Netflix. https://about.netflix.com/en/news/completing-the-netflix-cloud-migration